

5. Policy-Based Reinforcement Learning for Continuous Control Problems

Policy-Based Reinforcement Learning is a type of **Reinforcement Learning (RL)** in which the agent directly learns a **policy**. A policy tells the agent what action to take for a given state.

It is especially useful for **continuous control problems**, where actions are not limited to fixed choices. For example, a robot may need to decide the exact motor torque, and an autonomous car may need to decide the exact steering angle or acceleration.

1. Vanilla Policy Gradient

Definition

Vanilla Policy Gradient (VPG) is a basic policy-based reinforcement learning algorithm that directly optimizes the policy to maximize the expected reward.

Instead of learning a value function and then selecting an action, VPG learns the policy parameters directly.

Working

1. The agent observes the current state.
2. The policy generates an action.
3. The environment gives a reward.
4. The agent continues interacting with the environment.
5. The total reward is calculated.
6. The policy parameters are updated to increase the probability of actions that produced high rewards.
7. This process is repeated until the policy improves.

Policy

A policy can be represented as:

$$\pi(a|s)$$

where:

- s = current state
- a = action
- $\pi(a|s)$ = probability of selecting action a in state s

Advantages

- Simple and easy to understand.
- Directly learns the policy.
- Works well with continuous action spaces.
- Useful for robotics and autonomous systems.

Disadvantages

- Training can be unstable.
- Requires many interactions with the environment.
- Reward estimates can have high variance.
- May converge slowly.

Applications

- Robot arm control
- Autonomous vehicle steering
- Drone control
- Walking robots
- Continuous motor control

2. REINFORCE Algorithm

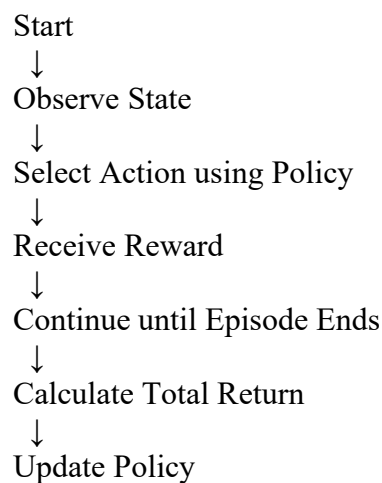
Definition

REINFORCE is a policy-gradient algorithm that learns a policy by using the rewards obtained from complete episodes.

It is one of the simplest algorithms for learning policies directly.

Working

The basic process is:



↓
Repeat

Steps

Step 1: Initialize policy

The agent starts with randomly initialized policy parameters.

Step 2: Select actions

The agent observes a state and selects an action according to the current policy.

Step 3: Collect rewards

The agent interacts with the environment and collects rewards until the episode ends.

Step 4: Calculate return

The return represents the total future reward obtained after taking an action.

Step 5: Update policy

Actions that produced higher rewards are made more likely in the future.

Simple idea

Suppose a robot performs two actions:

- Action A → high reward
- Action B → low reward

REINFORCE increases the probability of selecting **Action A** and decreases the probability of selecting **Action B**.

Advantages

- Simple implementation.
- Directly optimizes the policy.
- Suitable for stochastic policies.
- Can handle continuous actions.

Disadvantages

- High variance.
- Can require many training episodes.
- Learning may be slow.
- Performance depends heavily on reward design.

Applications

- Robot navigation
 - Autonomous driving
 - Game playing
 - Drone control
 - Robotic arm movement
-

3. Stochastic Policy Search

Definition

Stochastic Policy Search is a policy-based RL approach where the agent does not always select the same action for a particular state.

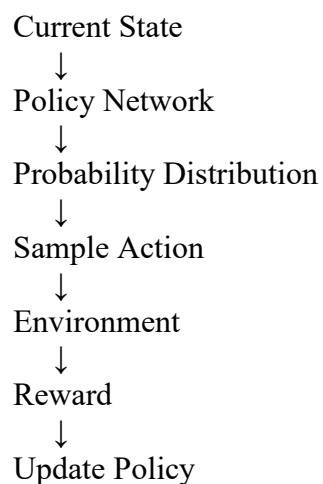
Instead, it uses a **probability distribution** to select actions.

For example, an autonomous vehicle approaching a turn might have:

Steer Left → 70%
Go Straight → 20%
Steer Right → 10%

The agent samples an action according to these probabilities.

Working



Continuous Control

For continuous actions, the policy can generate parameters of a distribution.

For example, a robot's motor control might be:

Motor action = 0.72

Instead of selecting only:

Left / Right

the agent can select different continuous values.

Advantages

- Supports exploration naturally.
- Suitable for uncertain environments.
- Works well with continuous actions.
- Can discover different possible solutions.

Disadvantages

- Training can be computationally expensive.
- Random actions can temporarily reduce performance.
- Requires careful reward design.
- May require many samples.

Applications

- Autonomous vehicles
 - Robotic arms
 - Drones
 - Industrial robots
 - Navigation systems
-

4. Applications in Robotics

Policy-based RL is highly useful in robotics because many robotic actions are **continuous**.

Example: Robot Arm

A robot arm needs to control:

- Joint angle
- Joint velocity
- Motor torque
- Movement direction

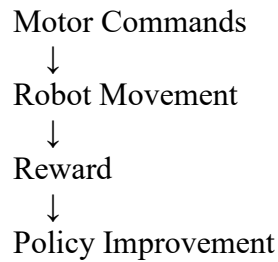
Instead of manually programming every movement, RL can learn a policy.

Robot State



Policy





Example

A robot arm is trained to pick up an object.

Positive reward:

- Successfully reaches object.
- Grasps object.
- Places object correctly.

Negative reward:

- Hits an obstacle.
- Drops the object.
- Takes too long.

Over multiple training episodes, the robot learns a better movement policy.

5. Applications in Autonomous Driving

Policy-based RL can be used to learn driving decisions and continuous vehicle control.

The state may contain:

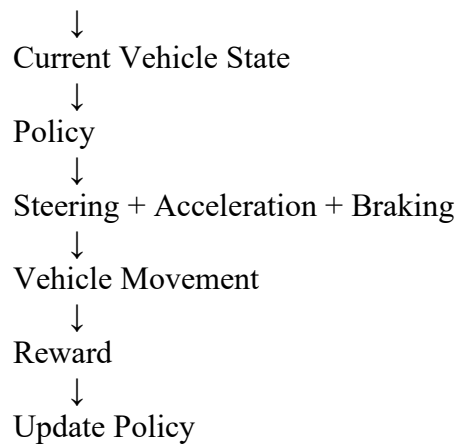
- Vehicle speed
- Road position
- Distance from other vehicles
- Traffic conditions
- Steering angle
- Sensor information

The actions may include:

- Steering angle
- Acceleration
- Braking

Working

Camera / Sensors



Reward Design

The autonomous vehicle can receive:

Positive rewards

- Staying in the lane
- Reaching the destination
- Maintaining safe speed
- Smooth driving

Negative rewards

- Collision
- Leaving the lane
- Sudden braking
- Excessive acceleration
- Long travel time

The objective is to learn a policy that provides **safe, efficient and smooth driving**.

6. Comparison of the Techniques

Technique	Main Idea	Suitable For
Vanilla Policy Gradient	Directly optimizes policy using rewards	Continuous control
REINFORCE	Updates policy using episode returns	Simple policy learning
Stochastic Policy Search	Samples actions from a probability distribution	Exploration and uncertainty
Robotics Application	Learns robot movements and motor control	Robot arms, drones, walking robots
Autonomous Driving	Learns steering, acceleration and braking	Self-driving vehicles

Real-Time Problem: Autonomous Car Steering Using Vanilla Policy Gradient

Problem

An **autonomous car** needs to learn how to control its steering while driving on a road. The car receives a state such as its distance from the center of the lane and chooses an action such as **left, straight, or right**.

The car gets:

- **+1 reward** when it moves closer to the lane center.
- **-1 reward** when it moves away from the lane center.

Using **Vanilla Policy Gradient**, the agent learns which action gives higher rewards.

```
import numpy as np

policy = np.array([0.33, 0.34, 0.33]) # Left, Straight, Right

for episode in range(10):
    action = np.random.choice(3, p=policy)
    reward = 1 if action == 1 else -1

    gradient = reward * (np.eye(3)[action] - policy)
    policy += 0.1 * gradient

    policy = np.maximum(policy, 0)
    policy /= policy.sum()

    print("Episode:", episode + 1, "Action:", action, "Reward:", reward)
```

Output

```
Episode: 1 Action: 1 Reward: 1
Episode: 2 Action: 1 Reward: 1
Episode: 3 Action: 0 Reward: -1
Episode: 4 Action: 1 Reward: 1
```

...

How it works

1. policy stores the probability of choosing **Left, Straight, Right**.
2. np.random.choice() selects an action according to these probabilities.
3. If the car chooses **Straight**, it gets +1.
4. Other actions receive -1.
5. The gradient calculates how the policy should change.
6. policy += 0.1 * gradient updates the policy.
7. The probabilities are normalized.
8. Over time, the probability of choosing the rewarding action increases.
9. Thus, the car learns a better driving policy.

10. This demonstrates the basic idea of **Vanilla Policy Gradient**.

1. Dynamic Programming for Sequential Decision-Making in Reinforcement Learning

Introduction

Dynamic Programming (DP) is a mathematical approach used in Reinforcement Learning to solve **sequential decision-making problems**. It helps an agent find the best actions by breaking a complex problem into smaller subproblems.

In Reinforcement Learning, Dynamic Programming is mainly used when the **environment model is known**, including the possible states, actions, transition probabilities, and rewards.

The main goal is to find an **optimal policy** that maximizes the total expected reward.

1. Bellman Principle of Optimality

The **Bellman Principle of Optimality** states that:

An optimal decision from the current state should also contain optimal decisions for all future states.

In simple words, the agent chooses the current action by considering the **best possible future rewards**.

The Bellman equation for the value function is:

$$V(s) = \max_a [R(s,a) + \gamma V(s')]$$

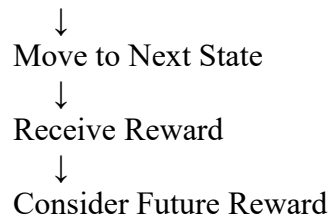
Where:

- $V(s)$ → Value of the current state
- $R(s,a)$ → Reward received after taking action
- γ → Discount factor
- $V(s')$ → Value of the next state
- a → Selected action

Example

A warehouse robot has to move from one location to another.

Current State
↓
Choose Action



The robot selects the action that provides the **highest expected future value**.

2. Policy Evaluation and Policy Improvement

Policy Evaluation

Policy Evaluation calculates how good the current policy is.

A policy tells the agent which action to take in each state.

For example:

State 1 → Move Right

State 2 → Move Up

State 3 → Move Right

The algorithm calculates the expected value of following this policy.

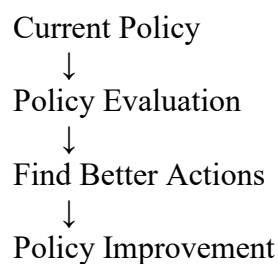
Purpose

- Determine the value of each state.
 - Measure the performance of the current policy.
 - Estimate future rewards.
-

Policy Improvement

After evaluating the policy, the agent checks whether another action can provide a better reward.

If a better action is found, the policy is updated.



↓
Improved Policy

This process continues until the policy becomes optimal.

3. Value Iteration vs Policy Iteration

Both **Value Iteration** and **Policy Iteration** are Dynamic Programming methods used to find an optimal policy.

Value Iteration

Value Iteration directly updates the value of each state using the Bellman optimality equation.

Initialize Values
↓
Update State Values
↓
Choose Best Action
↓
Repeat
↓
Optimal Policy

It focuses mainly on finding the **optimal value function**.

Policy Iteration

Policy Iteration alternates between two steps:

1. **Policy Evaluation**
2. **Policy Improvement**

Initialize Policy
↓
Policy Evaluation
↓
Policy Improvement
↓
Policy Stable?
↙ ↘
No Yes
↓ ↓
Repeat Optimal Policy

Comparison

Feature	Value Iteration	Policy Iteration
Main process	Updates values	Evaluates and improves policy
Policy evaluation	Implicit	Explicit
Policy improvement	After value update	Separate step
Implementation	Relatively simple	More computationally intensive per iteration
Goal	Optimal value function	Optimal policy

4. Industrial Applications

Dynamic Programming is used in many real-world applications where decisions need to be made sequentially.

1. Robotics

Robots can use DP for:

- Path planning
- Navigation
- Movement optimization
- Task scheduling

2. Autonomous Vehicles

DP can help vehicles make decisions related to:

- Route planning
- Speed control
- Lane selection
- Collision avoidance

3. Manufacturing

It can optimize:

- Production scheduling
- Machine operation
- Resource allocation
- Inventory management

Real-Time Problem: Robot Navigation Using Dynamic Programming

Problem

Consider a **warehouse robot** that needs to move from a starting position to a target position.

The robot can move:

- $0 \rightarrow \text{Left}$
- $1 \rightarrow \text{Right}$
- $2 \rightarrow \text{Up}$
- $3 \rightarrow \text{Down}$

The robot receives:

- **+10 reward** for reaching the goal.
- **-1 reward** for every movement.

Using **Dynamic Programming**, the robot learns the best action to reach the goal with minimum steps.

Simple 10-Line Python Code — Value Iteration

```
import numpy as np

V = np.zeros(4)
goal = 3
gamma = 0.9

for i in range(10):
    for state in range(3):
        actions = [state + 1]
        V[state] = -1 + gamma * max(V[a] for a in actions)

V[goal] = 10

for state in range(4):
    print("State:", state, "Value:", round(V[state], 2))
```

Sample Output

```
State: 0 Value: 5.39
State: 1 Value: 7.10
State: 2 Value: 8.00
State: 3 Value: 10.00
```

Simple Explanation

The states represent different positions of the robot:

Start $\rightarrow$ State 0 $\rightarrow$ State 1 $\rightarrow$ State 2 $\rightarrow$ Goal

The important line is:

$$V[\text{state}] = -1 + \text{gamma} * \max(V[a] \text{ for } a \text{ in actions})$$

It follows the **Bellman Principle of Optimality**:

The best decision at the current state depends on the best possible future value.

Here:

- -1 $\rightarrow$ movement cost
- gamma $\rightarrow$ discount factor
- $V[a]$ $\rightarrow$ value of the next state
- max() $\rightarrow$ chooses the best future action